

Merge Sort: Divide, Conquer, Combine

Programming and Data Structures — Week 6, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain the divide-and-conquer algorithmic strategy precisely, using merge sort as the concrete example; implement merge sort entirely from scratch, including the crucial merge step that combines two sorted halves; trace merge sort's complete recursion tree on a concrete numeric input; and prove that merge sort runs in $\Theta(n \log n)$ time in every case — best, average, and worst — using the recursion tree's structure directly.

Outline

Today covers a two-pile-of-sorted-cards analogy for merging; a precise definition of the divide-and-conquer strategy; the merge step, traced completely as its own operation; merge sort's full recursive behavior, traced completely on an 8-element input; the recursion tree and exactly where the $n \log n$ term comes from; merge sort's $O(n)$ extra space requirement, a real cost worth being honest about; the MTech-track iterative, bottom-up variant; an in-class quiz; and a summary.

The two-pile-of-sorted-cards analogy

Picture two piles of playing cards, each already sorted internally from smallest to largest, face up so the top card of each pile is visible. Merging them into one sorted pile is straightforward: repeatedly compare the two piles' current top cards, take whichever is smaller, and place it onto a new output pile, repeating until both piles are exhausted. This merging step is trivially efficient once both input piles are already sorted — the genuinely hard work was sorting each pile in the first place, which is exactly where merge sort's recursive structure comes in.

Divide and conquer, defined precisely

Divide and conquer is a general algorithmic strategy with three named steps. Divide breaks the original problem into smaller subproblems of identical structure. Conquer solves each subproblem, typically by applying the same strategy recursively, down to a base case simple enough to solve directly. Combine merges the subproblems' solutions back into a solution for the original problem. Merge sort is the canonical example: divide splits the array into two halves, conquer recursively sorts each half independently, and combine merges the two now-sorted halves using exactly the card-pile merging idea from the previous slide.

The merge step, in code

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])      # append whatever remains of 'left', if any
    result.extend(right[j:])    # append whatever remains of 'right', if any
    return result
```

The merge function walks both sorted input lists simultaneously with two pointers, *i* and *j*, always appending the smaller of the two current elements to the result. Using `<=` rather than strict `<` in the comparison is deliberate: when the two current elements are equal, the one from `left` is taken first, preserving the original relative order of equal elements — this single detail is what makes merge sort stable. Once either list is fully exhausted, the remainder of the other list is already sorted by construction and can simply be appended in full, with no further comparison needed.

Tracing the merge step on two sorted halves

`left = [2, 5, 8], right = [1, 3, 9]`

`i=0, j=0: left[0]=2 vs right[0]=1 → 1 smaller → result=[1], j=1`

`i=0, j=1: left[0]=2 vs right[1]=3 → 2 smaller → result=[1,2], i=1`

`i=1, j=1: left[1]=5 vs right[1]=3 → 3 smaller → result=[1,2,3], j=2`

$i=1, j=2$: $\text{left}[1]=5$ vs $\text{right}[2]=9 \rightarrow 5$ smaller $\rightarrow \text{result}=[1,2,3,5]$, $i=2$
 $i=2, j=2$: $\text{left}[2]=8$ vs $\text{right}[2]=9 \rightarrow 8$ smaller $\rightarrow \text{result}=[1,2,3,5,8]$, $i=3$
 $i=3$: left exhausted $\rightarrow$ append $\text{right}[2:] = [9] \rightarrow \text{result}=[1,2,3,5,8,9]$

Tracing the merge of $[2, 5, 8]$ and $[1, 3, 9]$ step by step: each comparison advances exactly one pointer and appends exactly one element to the result, continuing until left is exhausted after 5 comparisons, at which point the single remaining element of right, 9, is appended directly without further comparison. The merge step costs exactly $O(m + n)$ for two lists of length m and n — every element is examined and placed exactly once.

Merge sort itself, in code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr                # base case: 0 or 1 elements, already sorted
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])  # recursively sort the left half
    right = merge_sort(arr[mid:]) # recursively sort the right half
    return merge(left, right)     # combine
```

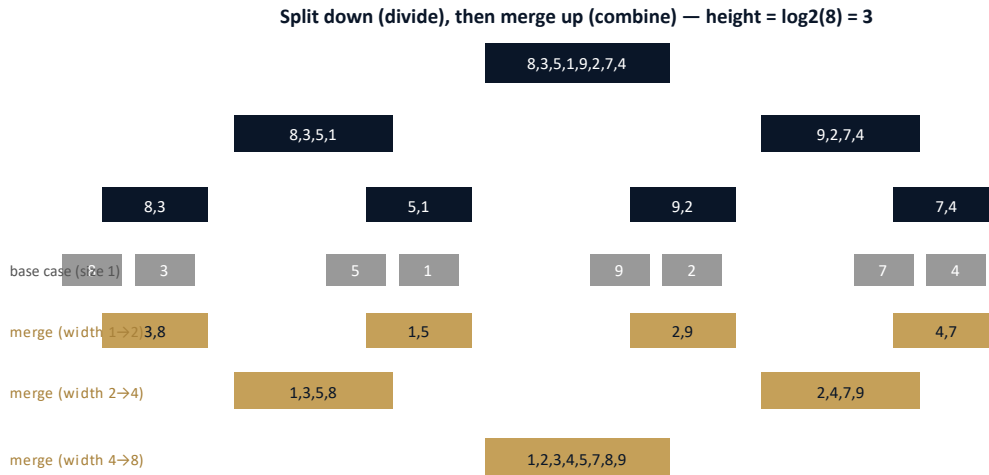
Merge sort's own code is remarkably short, precisely because it delegates all the real combining work to the merge function just traced. The base case handles arrays of 0 or 1 elements, which are trivially already sorted. Otherwise, the array splits at its midpoint, each half is sorted recursively via the exact same function, and the two now-sorted halves are combined using merge. This is divide-and-conquer applied directly: divide at the midpoint, conquer via recursion, combine via merging.

Tracing merge sort's full recursion on 8 elements

```
merge_sort([8,3,5,1,9,2,7,4])
  merge_sort([8,3,5,1])
    merge_sort([8,3])  $\rightarrow$  merge_sort([8]), merge_sort([3])  $\rightarrow$  merge([8],[3]) = [3,8]
    merge_sort([5,1])  $\rightarrow$  merge_sort([5]), merge_sort([1])  $\rightarrow$  merge([5],[1]) = [1,5]
     $\rightarrow$  merge([3,8],[1,5]) = [1,3,5,8]
  merge_sort([9,2,7,4])
    merge_sort([9,2])  $\rightarrow$  merge([9],[2]) = [2,9]
    merge_sort([7,4])  $\rightarrow$  merge([7],[4]) = [4,7]
     $\rightarrow$  merge([2,9],[4,7]) = [2,4,7,9]
 $\rightarrow$  merge([1,3,5,8],[2,4,7,9]) = [1,2,3,4,5,7,8,9]
```

Tracing the complete recursion on 8 elements shows the algorithm splitting all the way down to single elements — the base case — before any merging happens at all, then merging back upward in pairs: single elements merge into pairs of 2, pairs merge into groups of 4, and finally the two groups of 4 merge into the fully sorted result of 8. This “split all the way down, then merge all the way back up” pattern is the defining shape of merge sort’s execution, worth internalizing visually before the formal cost analysis.

The picture: the recursion tree



This recursion tree is the single most important image for understanding merge sort’s cost: the tree has $\log_2 n$ levels (splitting an n -element array in half repeatedly reaches size 1 after $\log_2 n$ halvings), and at every level, the total work across all the merges at that level is $O(n)$, since every element is touched exactly once per level regardless of how many pieces it has been split into.

Why $\Theta(n \log n)$: the recursion tree argument

The recursion tree has exactly $\log_2 n$ levels, since repeatedly halving n reaches 1 after $\log_2 n$ divisions. At every single level, the total work done across all the merge operations happening at that level is $O(n)$: whether there are 2 merges of size $n/2$ each, or 4 merges of size $n/4$ each, the total number of elements processed at that level is always n . Multiplying work-per-level by number-of-levels gives $O(n) \times O(\log n) = O(n \log n)$ total work. Critically, this argument does not depend on the input’s initial order at all — merge sort always splits exactly in half and always merges the full contents of both halves, so its cost is $\Theta(n \log n)$ in the best, average, *and* worst case alike, unlike every algorithm covered in Week 5.

Why merge sort is stable, revisited concretely

Consider a tie between a record (3, "x") sitting in `left` and a record (3, "y") sitting in `right`, where both compare equal on the sort key. Because `merge` uses `left[i] <= right[j]`, a tie resolves in favor of `left`, taking (3, "x") first. If "x" originated earlier than "y" in the array before it was split into `left` and `right` — which recursive splitting always preserves — this means the original relative order between tied elements survives every merge, all the way up the recursion tree. This is the concrete mechanism behind the general stability claim made earlier in this lecture.

Space cost: merge sort is not free

It is important to be honest about merge sort's cost beyond time. The `merge` function allocates a new list at every single call, and although these allocations happen at different levels of the recursion, the total extra memory used across the entire sort is $O(n)$ — proportional to the input size, not the $O(1)$ constant extra space that Week 5's in-place simple sorts required. This is a genuine trade-off: merge sort buys its guaranteed $\Theta(n \log n)$ time at the cost of $O(n)$ additional memory, a cost that matters when sorting extremely large datasets under tight memory constraints.

MTech addition — bottom-up (iterative) merge sort

```
def merge_sort_iterative(arr):
    arr = arr[:]
    n = len(arr)
    width = 1
    while width < n:
        for i in range(0, n, 2 * width):
            left = arr[i:i + width]
            right = arr[i + width:i + 2 * width]
            arr[i:i + len(left) + len(right)] = merge(left, right)
        width *= 2
    return arr
```

This bottom-up variant achieves the exact same result without any explicit recursion. It starts by treating every single element as an already-“sorted” run of width 1, merges adjacent runs into width-2 runs, then width-4, doubling the run width on each full pass over the array until the width reaches or exceeds n . This has the same $\Theta(n \log n)$ time and $O(n)$ space as the recursive version, but avoids relying on the recursion depth Python's call stack can support — genuinely useful for sorting very large arrays where deep recursion could otherwise be a practical concern, connecting directly back to Week 3's discussion of the call stack.

A second worked trace — 6 elements, uneven split

A second trace, on 6 elements rather than a clean power of 2, confirms the algorithm handles uneven splits correctly — `len(arr)//2` naturally produces a 3-and-3 split here, and within each half, a further 1-and-2 split. The recursion tree is not perfectly balanced in the sense of every leaf appearing at the same depth, but the total work per level remains $O(n)$ regardless, since the $\log_2 n$ -level argument only requires the array to roughly halve at each split, not to split into exactly equal integer halves every time.

Exercise

As an exercise, instrument the `merge` function to count comparisons made, run `merge_sort` on a random 16-element array, and confirm the total comparison count is close to the theoretical $16 \log_2 16 = 64$ predicted by this lecture's analysis. Separately, trace `merge_sort_iterative` by hand on `[5, 2, 8, 1]`, recording the full array contents after the `width=1` pass and again after the `width=2` pass, confirming it reaches the same fully sorted result as the recursive version.

Recurrence relations: the formal version of the tree argument

The recursion tree argument can be stated more formally as a recurrence relation: $T(n) = 2T(n/2) + O(n)$, where $T(n)$ is the total time to sort n elements. The $2T(n/2)$ term accounts for the two recursive calls, each operating on half the input; the $O(n)$ term accounts for the merge step's cost at the current level. Solving this recurrence — a general technique this course does not develop in full until dynamic programming's recursive structure in Week 9, though the Master Theorem is the standard tool — yields $T(n) = \Theta(n \log n)$, exactly matching the informal recursion-tree argument given earlier in this lecture. Both are valid, complementary ways of reaching the identical result.

Quiz — where does the cost come from?

For an array of one million elements, work out, before checking the answer, roughly how many levels merge sort's recursion tree has, and roughly how much total work happens at each individual level, using this lecture's recursion-tree analysis.

Quiz — answer

For $n = 1,000,000$, the number of levels is $\log_2(1,000,000) \approx 20$. Each level does $O(n)$ work, roughly 1,000,000 operations. Multiplying gives roughly 20,000,000 total operations — dramatically less than the roughly 10^{12} operations a naive $O(n^2)$ sort like selection sort would require on the same input, a difference of five orders of magnitude that makes the $n \log n$ versus n^2 distinction concrete rather than abstract.

Real-world use of merge sort

Merge sort is not merely a teaching exercise — Python’s Timsort, the algorithm behind `sorted()` and `list.sort()`, is fundamentally a hybrid of merge sort and the insertion sort covered in Week 5 Lecture 2, using insertion sort for small runs and merging them exactly as this lecture describes. Merge sort’s stability and guaranteed $\Theta(n \log n)$ time regardless of input make it the standard choice for external sorting, where data is too large to fit in memory and must be sorted in chunks read from and written back to disk — the merge step’s sequential access pattern is exactly what disk I/O rewards. A “merge join” in a database query engine, combining two sorted tables efficiently, is this exact merge operation applied directly to database records.

Summary

Merge sort divides an array in half, recursively conquers each half, and combines the results using a stable merge step. Its recursion tree has $\log_2 n$ levels, each performing $O(n)$ total work, giving $\Theta(n \log n)$ time in every case — the first algorithm in this course to actually achieve the $\Omega(n \log n)$ lower bound proven in Week 5. This guarantee costs $O(n)$ extra memory, a genuine trade-off against Week 5’s in-place simple sorts. Next lecture covers quicksort, which achieves the same $\Theta(n \log n)$ average-case time and, unlike merge sort, sorts in place — but without merge sort’s guaranteed worst case, a trade-off examined in full.